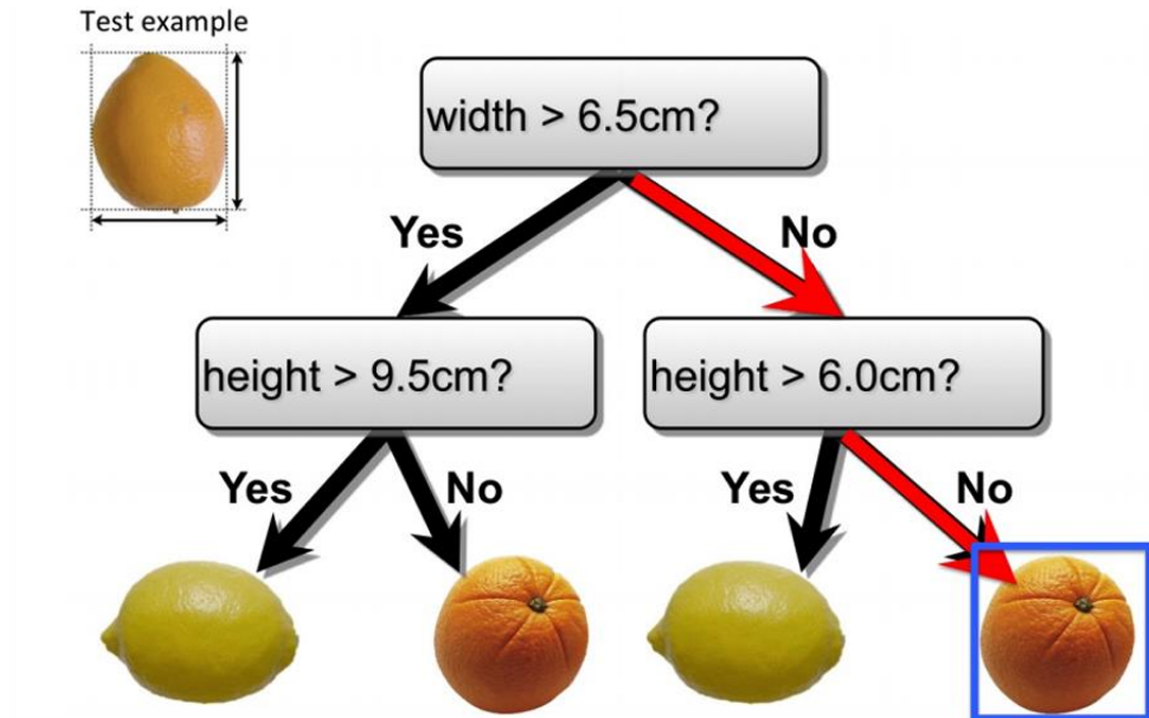


- [1. 决策树](#)
 - [1.1 连续特征 \(Continuous Features\)](#)
 - [1.2 决策树的组成](#)
 - [1.3 决策树在分类 \(Classification\) 和回归 \(Regression\) 任务中的应用](#)
 - [1.3 决策树的细节](#)
 - [1.3.1 贪心算法 \(greedy heuristic\)](#)
 - [1.3.2 熵 \(entropy\)](#)
 - [1.3.2.1 熵的特性](#)
 - [1.3.2.2 信息增益 \(Information Gain\)](#)
 - [1.3.3 构建决策树的完整贪心算法](#)
 - [1.3.4 构建决策树的指导原则](#)
 - [1.4 决策树、K近邻 \(KNN\) 和神经网络的对比](#)
- [2. 偏差方差分解](#)
 - [2.1 基本设置](#)
 - [2.2 贝叶斯最优性 \(Bayes Optimality\)](#)

1. 决策树

决策树是一种非常直观且易于理解的机器学习算法。它通过一系列的规则（通常是“如果-那么”的形式）来对数据进行分类或回归。尽管它看起来简单，但在很多情况下都能表现出强大的性能。

下面是一个决策树的例子用于判断水果是否是橘子。



宽度判断：首先检查水果的宽度是否大于6.5厘米。

如果是 (Yes)，则进入下一个判断。

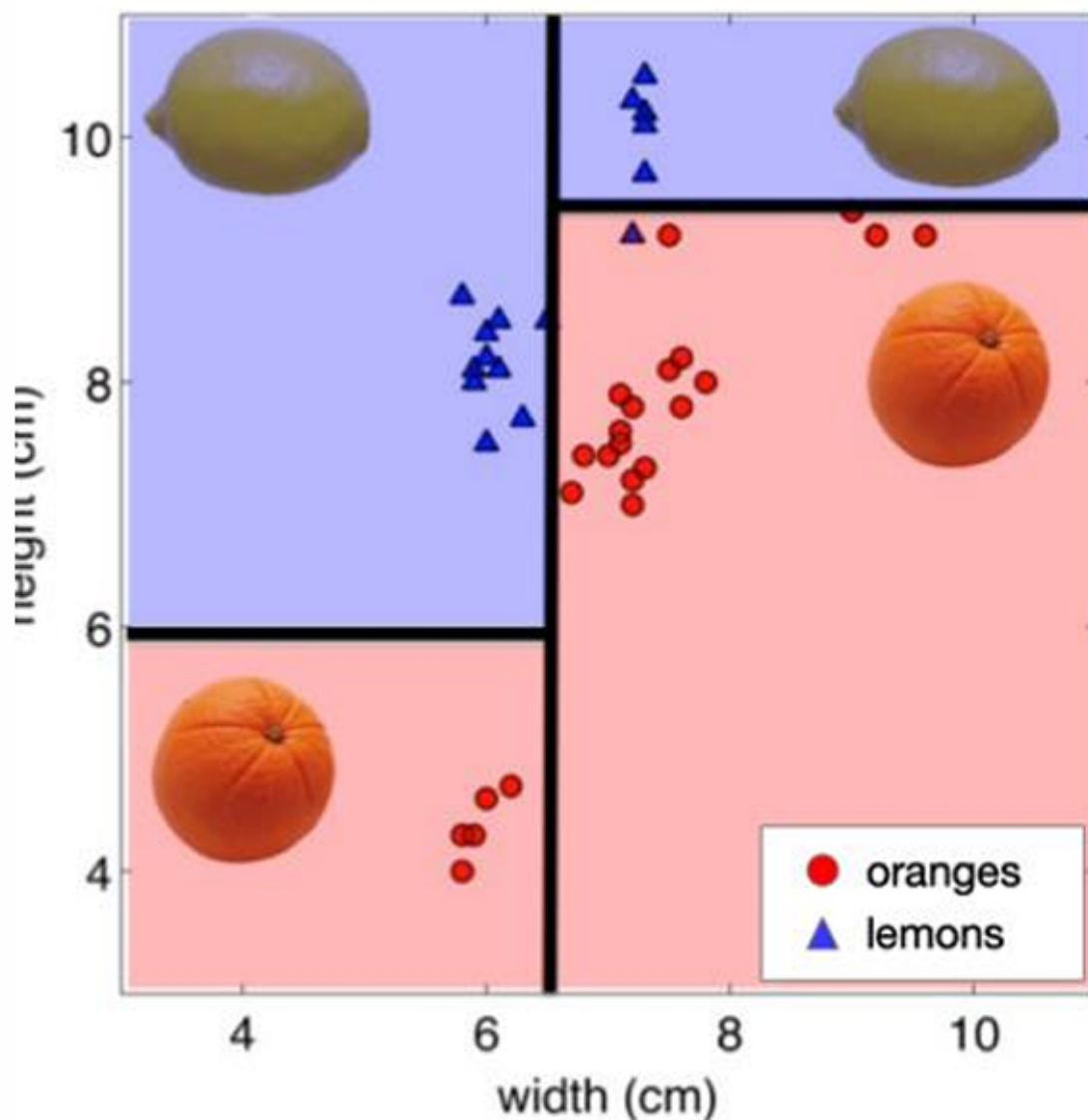
如果不是 (No)，则检查高度是否大于6.0厘米。

高度判断：

如果宽度大于6.5厘米，检查高度是否大于9.5厘米。
如果是（Yes），则分类为柠檬（图中左侧的柠檬）。
如果不是（No），则分类为橙子（图中中间的橙子）。
如果宽度不大于6.5厘米，检查高度是否大于6.0厘米。
如果是（Yes），则分类为柠檬（图中右侧的柠檬）。
如果不是（No），则分类为橙子（图中右下角的橙子）。

1.1 连续特征（Continuous Features）

决策树通过检查特征是否大于或小于某个阈值来分割连续特征。
决策树的决策边界由轴对齐的平面组成，这意味着边界是垂直于特征轴的直线。



1.2 决策树的组成

决策树包含三个部分：

1. 内部节点（Internal nodes）：这些节点用于测试特征。在这个例子中，内部节点测试的是水果的宽度和高度。
2. 分支（Branching）：分支由特征值决定。如果特征值满足某个条件（例如，宽度大于6.5厘米），则沿着“是”（Yes）的分支继续；如果不满足，则沿着“否”（No）的分支继续。
3. 叶节点（Leaf nodes）：这些节点是输出（预测结果）。在这个例子中，叶节点表示最终的分类结果，即橙子或柠檬。

1.3 决策树在分类 (Classification) 和回归 (Regression) 任务中的应用

分类树 (Classification Tree) :

离散输出 (Discrete Output) : 分类树用于预测离散的类别标签。

叶节点值设置: 叶节点的值 y^m 通常设置为该叶节点下所有训练样本中最常见的类别标签。这意味着, 如果叶节点包含的训练样本中, 橙子的数量多于柠檬, 那么该叶节点的值将被设置为“橙子”。

回归树 (Regression Tree) :

连续输出 (Continuous Output) : 回归树用于预测连续的数值。

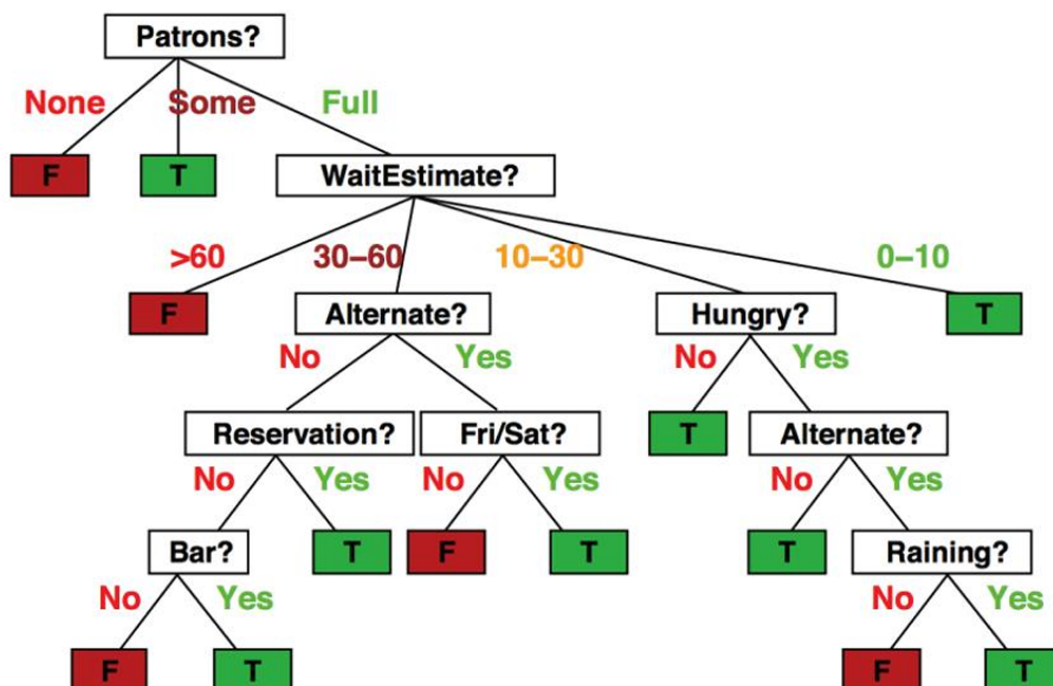
叶节点值设置: 叶节点的值 y^m 通常设置为该叶节点下所有训练样本的目标值的平均值。这意味着, 如果叶节点包含的训练样本的目标值是一组连续的数值, 那么该叶节点的值将被设置为这些数值的平均值。

两者的共同点:

路径定义区域: 从根节点到叶节点的每条路径定义了输入空间的一个区域 R^m 。

训练样本: 设 $\{(x^{(m_1)}, t^{(m_1)}), \dots, (x^{(m_k)}, t^{(m_k)})\}$ 为落入区域 R^m 的训练样本集合, 其中 x 表示特征, t 表示目标值。

下图又示范了一个是否在餐厅等待晚餐的例子。



将离散特征分割成可能值的分区（Split discrete features into a partition of possible values）。

Example	Input Attributes										Goal WillWait
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	
x ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	y ₁ = Yes
x ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	y ₂ = No
x ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	y ₃ = Yes
x ₄	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	y ₄ = Yes
x ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	y ₅ = No
x ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	y ₆ = Yes
x ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	y ₇ = No
x ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	y ₈ = Yes
x ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	y ₉ = No
x ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	y ₁₀ = No
x ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	y ₁₁ = No
x ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	y ₁₂ = Yes

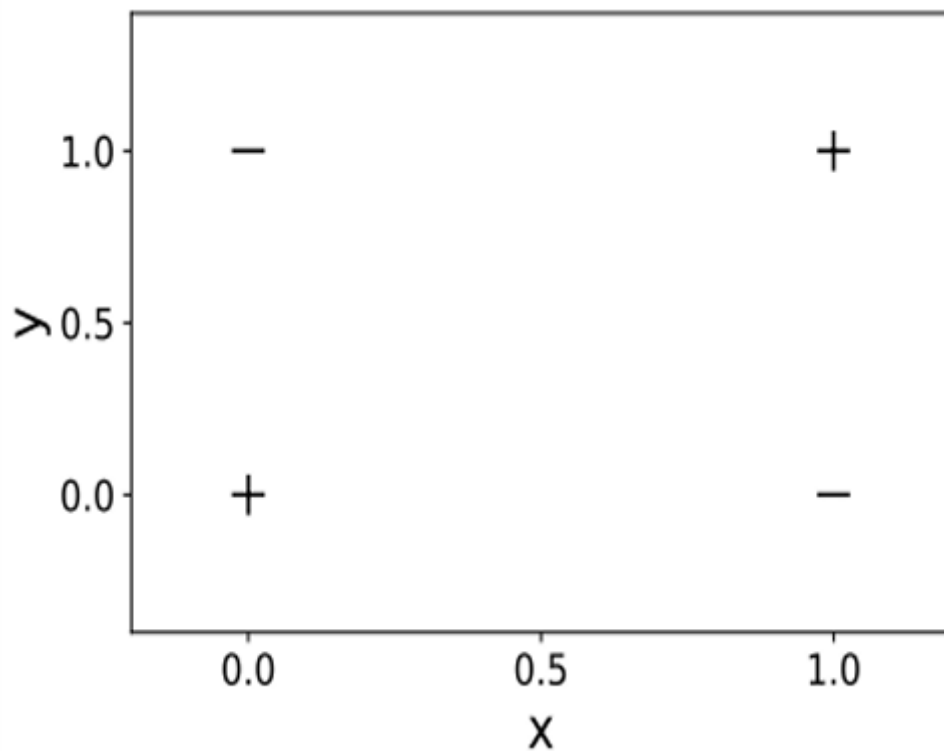
1.	Alternate: whether there is a suitable alternative restaurant nearby.
2.	Bar: whether the restaurant has a comfortable bar area to wait in.
3.	Fri/Sat: true on Fridays and Saturdays.
4.	Hungry: whether we are hungry.
5.	Patrons: how many people are in the restaurant (values are None, Some, and Full).
6.	Price: the restaurant's price range (\$, \$\$, \$\$\$).
7.	Raining: whether it is raining outside.
8.	Reservation: whether we made a reservation.
9.	Type: the kind of restaurant (French, Italian, Thai or Burger).
10.	WaitEstimate: the wait estimated by the host (0-10 minutes, 10-30, 30-60, >60).

Features:

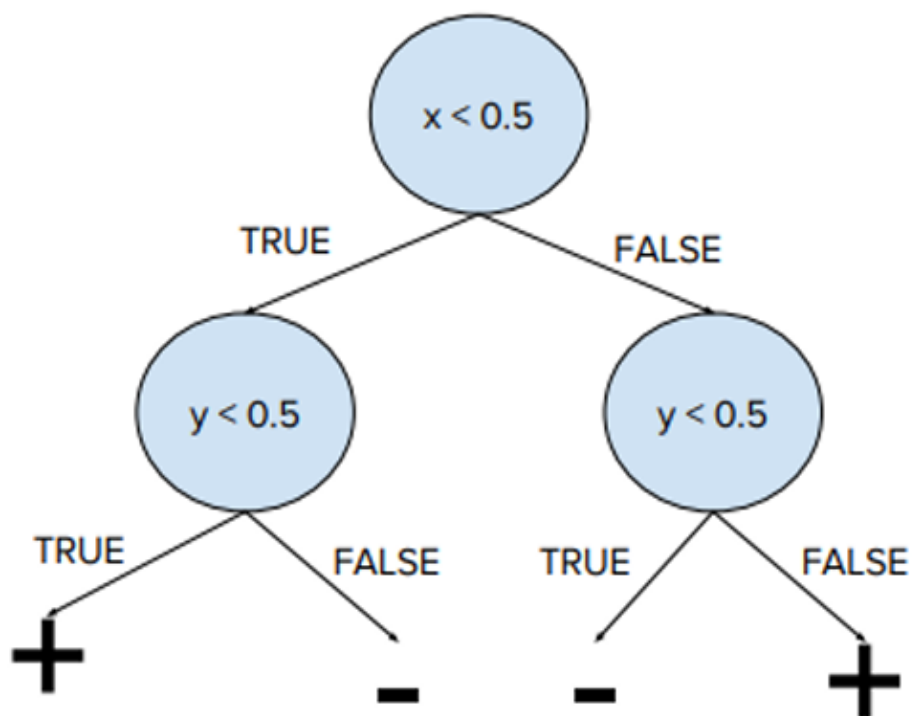
这样我们使用决策树处理离散特征来预测是否会在餐厅等待晚餐。

1.3 决策树的细节

我们先从一个简单示例开始。



每个样本有两个输入特征 x 和 y ，并被分类为正例（标记为+）或负例（标记为-）。我们要绘制一个决策树，该决策树能够正确地对数据集中的每个样本进行分类。这里构建决策树很简单，这里给出一个示例。



当然也可以先用 y 值区分，再用 x 值区分。

其实对于任何训练集，我们都可以构建一个决策树，但该树为每个训练点恰好有一个叶节点，但这棵树可能不会泛化（即在未见过的数据上表现不好）。

所以决策树是通用函数逼近器，这意味着理论上，它们可以逼近任何函数，只要树足够复杂。

找到正确分类训练集的最小决策树是一个NP完全问题。这意味着没有已知的多项式时间算法可以解决这

个问题，而且随着数据集的增大，找到最优解的计算量会急剧增加。
由于找到最小决策树是NP完全的，我们需要采取其他策略来构建有用的决策树。

1.3.1 贪心算法 (greedy heuristic)

贪心算法构建决策树：

开始：从整个训练集开始，构建一个空的决策树。

选择特征和分割点：选择一个特征和一个候选分割点，这个分割点能够最大程度地减少损失 (loss)。

分割数据：根据选定的特征和分割点对数据进行分割，并递归地对分割后的数据子集重复这个过程。

因此在决策树学习中，选择合适的损失函数是非常重要的，因为它直接影响到模型的性能和泛化能力。
误分类率是指模型预测错误的样本数占总样本数的比例。这是一个直观的损失函数，因为它直接反映了模型的分类性能。我们现在尝试用这个作为损失函数。

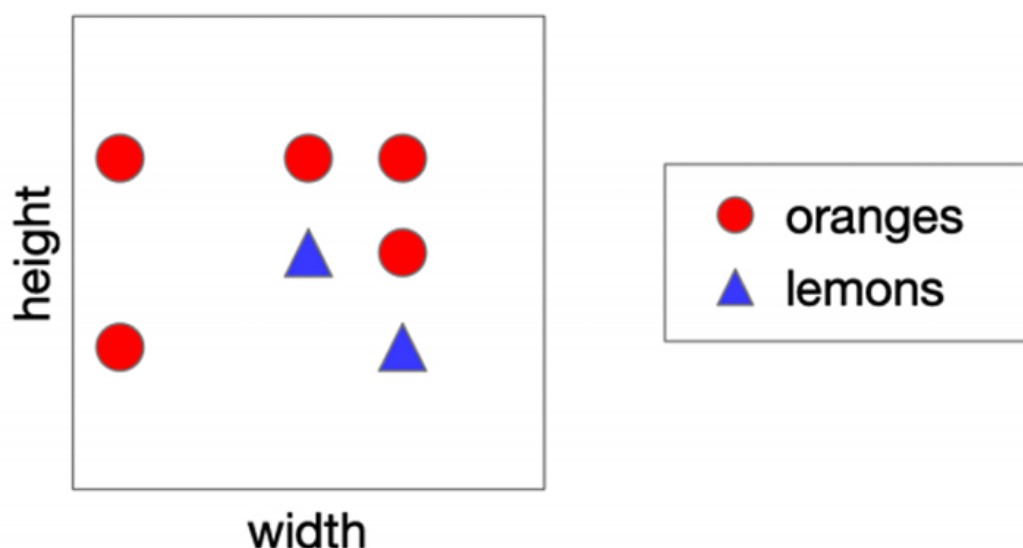
然而，误分类率可能不是最优的损失函数，原因包括：

不平衡数据：如果数据集中的类别分布非常不均匀，误分类率可能会过高估计少数类的分类性能。

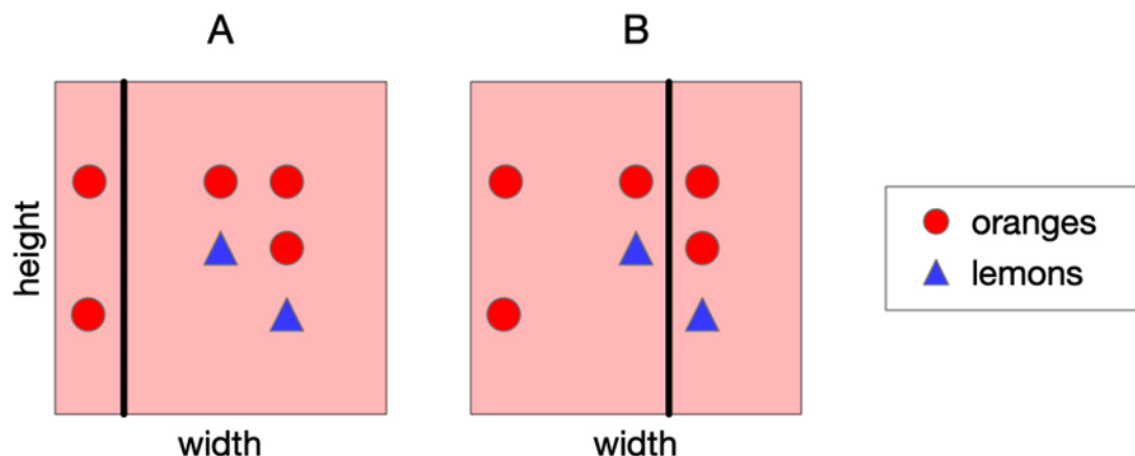
成本敏感性：在某些应用中，不同类型的错误可能有不同的成本。例如，在医疗诊断中，将患者错误地分类为健康可能比将健康的人分类为患者更严重。误分类率没有考虑到这种成本敏感性。

模型复杂性：单纯追求低误分类率可能导致过拟合，即模型在训练数据上表现很好，但在新数据上表现不佳。

我们看下面这个例子。



我们先尝试用宽度去分割。



这里A分割方案和B分割方案有相同的误分类率，那么哪个是最佳分割？

这里A似乎是一个更好的分割，因为左手区域非常确定水果是否是橙子。

在选择分割时，我们希望找到一个能够最大程度地减少误分类的分割点。方案A在左手区域中橙子的比例非常高，这使得该区域对分类结果非常确定。

我们希望我们的分割可以是减少叶节点不确定性的。

如果叶节点中的所有样本都属于同一个类别，那么这个叶节点的预测是确定的，不确定性低。这种情况下，模型对这个叶节点的预测非常有信心。

如果叶节点中每个类别的样本数量相同，那么这个叶节点的预测是不确定的，不确定性高。这种情况下，模型无法确定哪个类别是正确的，因为每个类别的可能性相同。

因此我们可以使用概率分布，使用叶节点中的样本计数来定义概率分布。这意味着每个类别在叶节点中出现的概率可以表示为该类别样本数与叶节点中总样本数的比例。

1.3.2 熵 (entropy)

我们使用信息论里的熵 (entropy) 来量化不确定性。

熵是一个数值，用于量化离散随机变量在其可能结果中固有的不确定性。

虽然熵的数学定义可能看起来是任意的，但它可以通过一组公理 (axioms) 来合理化。

我们用抛硬币的例子来解释这个概念。

我们现在有两次抛硬币，结果如下图所i示。

Sequence 1:

0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 ... ?

Sequence 2:

0 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 0 1 ... ?



熵的计算公式为: $H(p) = -p \log_2(p) - (1 - p) \log_2(1 - p)$

我们可以用这个公式计算硬币结果的不确定性。熵越高，表示结果的不确定性越大；熵越低，表示结果的不确定性越小。

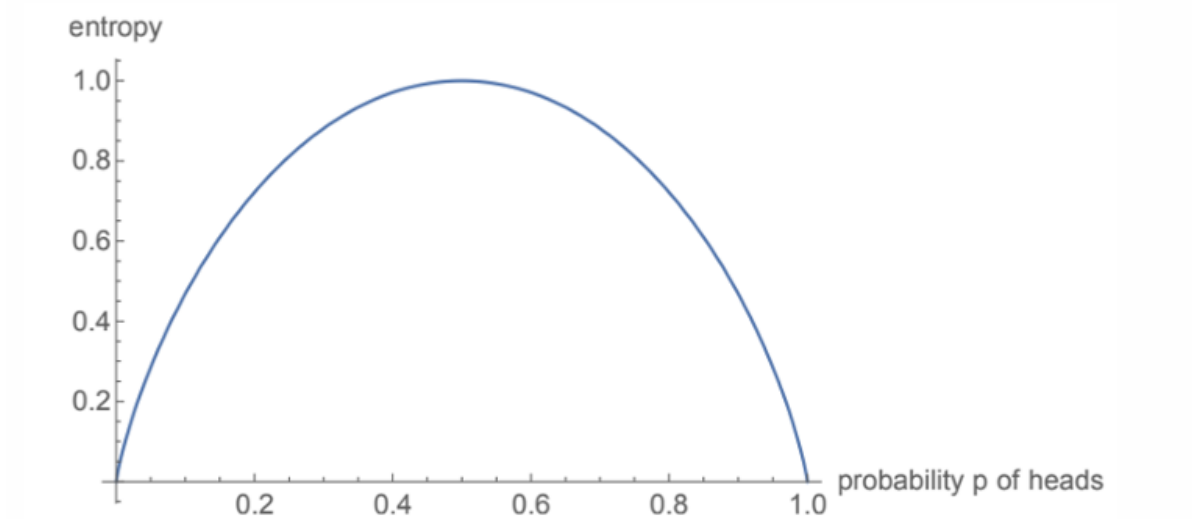
对于第一次: $H\left(\frac{1}{9}\right) = -\frac{8}{9} \log_2\left(\frac{8}{9}\right) - \frac{1}{9} \log_2\left(\frac{1}{9}\right) \approx 0.52$

对于第二次: $H\left(\frac{5}{9}\right) = -\frac{4}{9} \log_2\left(\frac{4}{9}\right) - \frac{5}{9} \log_2\left(\frac{5}{9}\right) \approx 0.99$

结果越确定（即概率 p 接近0或1），熵越低。这是因为在极端情况下（ $p = 0$ 或 $p = 1$ ），我们对结果已经非常确定，观察它不会增加任何新的信息，因此熵为0。

所以对于一个完全公平的硬币（即正面和反面出现的概率都是0.5），熵是最大的。这是因为在抛掷之前，我们对结果没有任何先验知识，结果是完全随机的，因此不确定性最高。

下图也体现了这一点。



可以将熵视为从概率分布中随机抽取的期望信息内容。这意味着熵衡量了在观察结果之前，我们对结果的不确定性。

信息论奠基者克劳德·香农说：你不能使用少于熵所表示的期望比特数来存储随机抽取的结果而不丢失信息。

熵的单位是比特（bits）。对于一个完全公平的硬币（正面和反面的概率都是0.5），其熵为1比特。这表示在观察结果之前，我们对结果的不确定性是1比特。

熵的通用定义公式为： $H(Y) = - \sum_{y \in Y} p(y) \log_2 p(y)$ ，其中

Y ：离散随机变量（比如类别、结果、标签等）

$p(y)$ ：变量取某个具体值 y 的概率

熵的单位为比特（bits）

高熵：接近均匀分布（每个值概率差不多），直方图平坦（flat），更难预测。

低熵：集中在少数值上（某一两个值概率很高），尖锐（peaked），更容易预测。

假设我们观察到关于随机变量 Y 的部分信息 X 。这里的 X 可以是 Y 的任何函数，例如 $X = \text{sign}(Y)$ ，即 X 表示 Y 的符号（正或负）。

我们希望定义通过观察 X 而获得的关于 Y 的期望信息量。这可以理解为观察 X 后，我们对 Y 的了解增加了多少。

或者等效地说，我们希望度量在观察 X 后，我们对 Y 的不确定性减少了多少。

初始熵 $H(Y)$ 表示在没有观察 X 之前对 Y 的不确定性。

条件熵 $H(Y | X)$ 表示在观察 X 之后对 Y 的剩余不确定性。

信息增益（Information Gain）定义为 $H(Y) - H(Y | X)$ ，即通过观察 X 减少的不确定性。

我们通过例子尝试理解一下这段话。下面的例子中随机变量 X 表示是否下雨， Y 表示是否多云。

	Cloudy	Not Cloudy
Raining	24/100	1/100
Not Raining	25/100	50/100

联合熵 (Entropy of a Joint Distribution) : $H(X, Y) = - \sum_{x \in X} \sum_{y \in Y} p(x, y) \log_2 p(x, y)$
 $= - \left(\frac{24}{100} \log_2 \frac{24}{100} + \frac{1}{100} \log_2 \frac{1}{100} + \frac{25}{100} \log_2 \frac{25}{100} + \frac{50}{100} \log_2 \frac{50}{100} \right)$
 $\approx 1.56 \text{ bits}$

从联合分布的熵 (1.56 bits) 可以看出，同时预测是否下雨和是否多云的不确定性较高，这表明直接预测天气 (包括下雨和多云) 是相对困难的。

条件熵 (Specific Conditional Entropy) : 我们现在计算一下正在下雨的情况下，多云的熵是多少？我们这里需要先计算下雨的情况下多云与不多云的概率，分别是 $\frac{24}{25}$ 和 $\frac{1}{25}$ 。

$H(Y|X = x) = - \sum_{y \in Y} p(y|x) \log_2 p(y|x)$
 $= - \left(\frac{24}{25} \log_2 \frac{24}{25} + \frac{1}{25} \log_2 \frac{1}{25} \right)$
 $\approx 0.24 \text{ bits}$

从条件熵 (0.24 bits) 可以看出，在已知下雨的情况下，预测是否多云的不确定性显著降低。

条件熵的期望值公式：

$$H(Y|X) = \sum_{x \in X} p(x) H(Y|X = x) = - \sum_{x \in X} \sum_{y \in Y} p(x, y) \log_2 p(y|x)$$

这个公式表示在已知 X 的情况下， Y 的条件熵是有可能 X 值上的条件熵的加权平均。

给定是否下雨的信息，计算多云的条件熵：

$$H(Y|X) = \sum_{x \in X} p(x) H(Y|X = x) = \frac{1}{4} H(\text{cloudiness}|\text{is raining}) + \frac{3}{4} H(\text{cloudiness}|\text{not raining})$$

$\approx 0.75 \text{ bits}$

这个公式表示在已知是否下雨的情况下，多云的条件熵是下雨时和不下雨时条件熵的加权平均。

我们可以看到在已知是否下雨的情况下，多云的条件熵 (0.75 bits) 比联合熵 (1.56 bits) 要低，这表明在已知是否下雨的情况下，我们对是否多云的预测会更加确定。

联合熵衡量两个随机变量 X 和 Y 同时的不确定性。联合熵提供了对两个变量同时的不确定性的全面度量，不考虑任何条件信息。

条件熵衡量在已知一个随机变量 X 的情况下，另一个随机变量 Y 的不确定性。特定条件熵关注在已知一个变量的特定值时，另一个变量的不确定性。

期望条件熵衡量在已知随机变量 X 的所有可能值的情况下，随机变量 Y 的平均不确定性。期望条件熵则考虑了所有可能的 X 值，提供了一个平均的不确定性度量，这有助于理解在不同条件下 Y 的平均不确定性。

1.3.2.1 熵的特性

1. 熵 H 的值永远不会是负数。
2. 链式法则表明，两个随机变量 X 和 Y 的联合熵可以表示为 X 的条件熵加上 Y 的熵，也可以表示为 Y 的条件熵加上 X 的熵，即 $H(X, Y) = H(X|Y) + H(Y) = H(Y|X) + H(X)$
3. 如果 X 和 Y 是独立的，那么知道 X 不会影响我们对 Y 的不确定性，即 $H(Y|X) = H(Y)$ 。
4. 如果我们知道 Y 的值，那么我们对 Y 的不确定性完全消失，即 $H(Y|Y) = 0$ 。
5. 通过知道 X ，我们只能减少对 Y 的不确定性，即 $H(Y|X) \leq H(Y)$ 。

1.3.2.2 信息增益 (Information Gain)

正如我们上面提到的通过知道 X ，我们只能减少对 Y 的不确定性。

信息增益衡量的是在观察到随机变量 X 后，随机变量 Y 的不确定性减少了多少。换句话说，它衡量了 X 对 Y 的信息量。

公式: $IG(Y|X) = H(Y) - H(Y|X)$ ，其中

$H(Y)$ 是 Y 的熵，表示在没有观察 X 之前对 Y 的不确定性。

$H(Y|X)$ 是在已知 X 的情况下 Y 的条件熵，表示在观察 X 之后对 Y 的剩余不确定性。

如果 X 对 Y 完全无信息，即 X 和 Y 独立，那么 $IG(Y|X) = 0$ 。

如果 X 完全确定了 Y ，即知道 X 后 Y 没有不确定性，那么 $IG(Y|X) = H(Y)$ 。

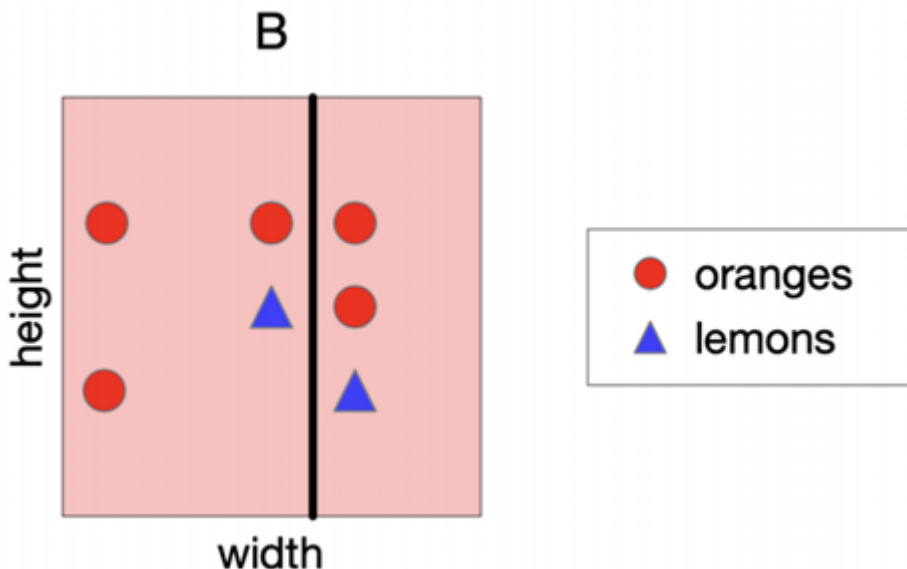
因为信息增益衡量一个变量（如特征）的信息量，即知道这个变量的值后，我们对目标变量（如类别标签）的不确定性减少了多少。

所以在构建决策树时，我们希望选择那些能够最大程度减少目标变量不确定性的特征进行分割。信息增益正是衡量这种减少不确定性的指标。

分割的信息增益指的是通过知道分割的哪一边（即通过某个特征的某个阈值分割数据后，数据落在哪个子集中），我们对目标变量（类别标签）的了解增加了多少信息。

信息增益是基于训练集计算的，它告诉我们在训练数据上，通过某个特征的分割，我们对目标变量的预测能力提高了多少。

因此回到我们前面的那个例子，分隔方案 B 的信息增益是多少？



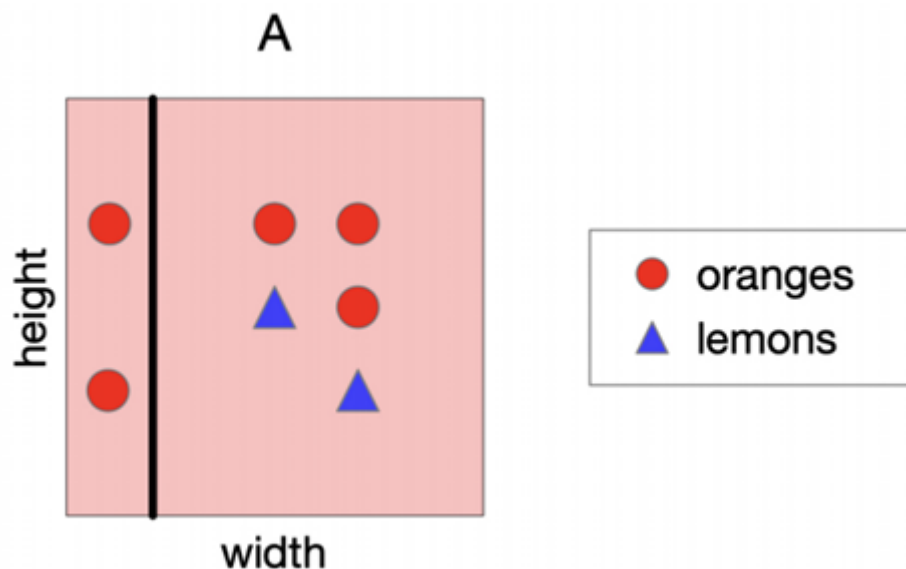
根节点的类别熵: $H(Y) = -\frac{2}{7}\log_2\left(\frac{2}{7}\right) - \frac{5}{7}\log_2\left(\frac{5}{7}\right) \approx 0.86$

左叶节点的条件熵: $H(Y|\text{left}) \approx 0.81$

右叶节点的条件熵: $H(Y|\text{right}) \approx 0.92$

分割 B 的信息增益: $IG(\text{split B}) \approx 0.86 - \left(\frac{4}{7} \cdot 0.81 + \frac{3}{7} \cdot 0.92\right) \approx 0.006$

那分隔方案A的信息增益是多少？



根节点的类别熵: $H(Y) = -\frac{2}{7}\log_2\left(\frac{2}{7}\right) - \frac{5}{7}\log_2\left(\frac{5}{7}\right) \approx 0.86$

左叶节点的条件熵: $H(Y|\text{left}) \approx 0$

右叶节点的条件熵: $H(Y|\text{right}) \approx 0.97$

分割 B 的信息增益: $IG(\text{split A}) \approx 0.86 - \left(\frac{2}{7} \cdot 0 + \frac{5}{7} \cdot 0.97\right) \approx 0.17$

分割A的信息增益 (0.17) 远大于分割B的信息增益 (0.006), 这意味着分割A在减少目标变量不确定性方面更有效, 因此在构建决策树时, 分割A是更好的选择。

1.3.3 构建决策树的完整贪心算法

现在我们给出完整的贪心算法以构建决策树:

1. 选择特征进行分割:

在非终端节点 (即决策树中的内部节点) 选择一个特征进行分割。这个特征的选择通常基于某些标准, 如信息增益、基尼指数等, 以最大化减少目标变量的不确定性。

2. 根据特征值分组样本:

根据所选特征的值将样本分割成不同的组。例如, 如果选择的特征是“宽度”, 那么可以根据宽度是否大于某个阈值将样本分为两组。

3. 对每个组进行处理:

对于每个分割后的组, 执行以下操作:

如果组内没有样本 (即该组为空), 则返回其父节点中样本的多数类。

如果组内所有样本都属于同一类, 则该节点成为叶节点, 并返回该类。

如果组内样本属于多个类, 则递归地重复步骤1和2, 即在该节点上选择新的特征进行分割。

4. 终止条件:

算法终止的条件是所有叶节点都只包含同一类的样本, 或者叶节点为空。这意味着决策树已经完全构建, 无法再进行进一步的分割。

我们再回到我们之前提到的一个例子。

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	$y_1 = \text{Yes}$
x_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	$y_2 = \text{No}$
x_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10	$y_3 = \text{Yes}$
x_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	$y_4 = \text{Yes}$
x_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	$y_5 = \text{No}$
x_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	$y_6 = \text{Yes}$
x_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	$y_7 = \text{No}$
x_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	$y_8 = \text{Yes}$
x_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	$y_9 = \text{No}$
x_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	$y_{10} = \text{No}$
x_{11}	No	No	No	No	None	\$	No	No	Thai	0-10	$y_{11} = \text{No}$
x_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	$y_{12} = \text{Yes}$

1.	Alternate: whether there is a suitable alternative restaurant nearby.
2.	Bar: whether the restaurant has a comfortable bar area to wait in.
3.	Fri/Sat: true on Fridays and Saturdays.
4.	Hungry: whether we are hungry.
5.	Patrons: how many people are in the restaurant (values are None, Some, and Full).
6.	Price: the restaurant's price range (\$, \$\$, \$\$\$).
7.	Raining: whether it is raining outside.
8.	Reservation: whether we made a reservation.
9.	Type: the kind of restaurant (French, Italian, Thai or Burger).
10.	WaitEstimate: the wait estimated by the host (0-10 minutes, 10-30, 30-60, >60).

Features:

我们以其中Type和Patrons这两个特征为例进行分隔：

$$IG(\text{type}) = 1 - \left[\frac{2}{12} H(Y|\text{Fr.}) + \frac{2}{12} H(Y|\text{It.}) + \frac{4}{12} H(Y|\text{Thai}) + \frac{4}{12} H(Y|\text{Bur.}) \right] = 0$$

$$IG(\text{Patrons}) = 1 - \left[\frac{2}{12} H(0, 1) + \frac{4}{12} H(1, 0) + \frac{6}{12} H\left(\frac{2}{6}, \frac{4}{6}\right) \right] \approx 0.541$$

Patrons特征比Type特征提供了更多的信息，因此在构建决策树时，Patrons是一个更好的分割选择。

1.3.4 构建决策树的指导原则

好的决策树需要足够大，以便能够处理数据中重要但可能微妙的区别。如果树太小，它可能无法捕捉到数据中的细微差别，从而导致模型过于简单，无法准确预测。

同时也不应该太大，原因包括：

计算效率：大的树可能会包含冗余或无关紧要的特征，这会影响计算效率。

避免过拟合：过拟合是指模型在训练数据上表现很好，但在新数据上表现不佳。大的树更容易记住训练数据的细节，而不是学习到泛化的特征。

可解释性：树越大，越难以理解和解释其决策过程。

我们应该寻找能够最好地拟合数据的最简单的树结构。

总结一下我们希望得到的决策树：

小树：我们希望树尽可能小，以提高计算效率和可解释性。

信息丰富的节点：树的根节点附近应该有信息量丰富的特征，这样可以更快地做出决策。

靠近根节点的节点：重要的特征应该在树的较高层次上被使用，这样可以在决策过程中更早地进行关键的分割。

所以决策树常见的问题如下：

1. 数据分布不均：

在决策树的较低层次，可用的数据量呈指数级减少。这意味着随着树的深度增加，每个叶节点所包含的样本数量会越来越少，这可能导致模型在这些节点上的预测不够稳定或准确。

2. 过拟合风险：

如果决策树过大，它可能会过拟合训练数据，即模型在训练数据上表现很好，但在未见过的数据上表现不佳。过拟合通常发生在模型过于复杂，以至于它捕捉到了数据中的噪声而非潜在的模式。

3. 局部最优而非全局最优：

决策树构建通常使用贪心算法（如信息增益），这些算法在每一步都选择局部最优的特征进行分割，但这并不保证最终得到的是全局最优的树结构。

对于连续属性（如年龄、温度等），决策树需要基于某个阈值进行分割。

选择阈值的目标是最大化信息增益，即选择一个阈值，使得分割后的子节点能够最大程度地减少不确定性或增加信息量。

1.4 决策树、K近邻（KNN）和神经网络的对比

决策树相对于KNN和神经网络的优势：

1. 处理离散特征、缺失值和未标准化数据：

决策树能够很好地处理分类特征、缺失值和未标准化的数据，而无需进行复杂的数据预处理。

2. 测试速度快：

决策树在测试时非常快，因为它们只需要沿着树从根到叶进行简单的路径查找。

3. 可解释性高：

决策树的结构直观，易于理解和解释，这对于需要模型解释的应用场景非常有用。

KNN相对于决策树的优势：

1. 超参数少：

KNN模型的超参数较少，主要是选择K值（邻居的数量）和距离度量，这使得模型调参相对简单。

2. 可以结合有趣的距离度量：

KNN可以利用各种距离度量（如欧氏距离、曼哈顿距离等），甚至可以结合形状上下文等复杂的度量方法，这为模型提供了灵活性。

神经网络相对于决策树的优势：

1. 能够处理高度交互的特征：

神经网络擅长处理特征之间复杂交互的情况，例如图像中的像素点，这些特征在决策树中很难有效处理。

它们之间各有优势，那我们可以试着将多个分类器（predictors）组合成一个集成，这些分类器的个体决策以某种方式结合起来，对新的样本进行分类。这便是集成学习（Ensemble Learning）。

为了使集成学习有意义（nontrivial），这些分类器必须在某些方面有所不同：

1. 不同的算法：使用不同的分类算法，如决策树、神经网络、支持向量机等。

2. 不同的超参数选择：即使使用相同的算法，也可以通过选择不同的超参数来创建不同的模型。

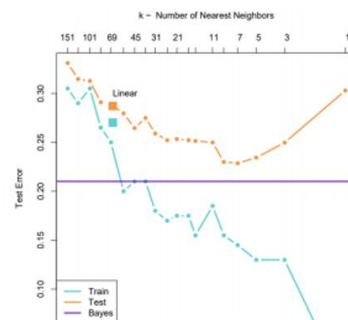
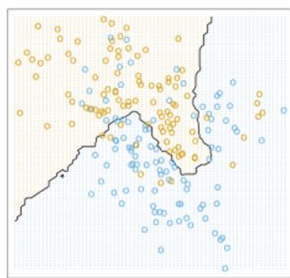
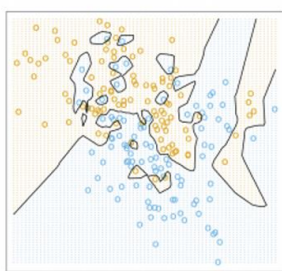
3. 在不同的数据上训练：在不同的数据子集上训练模型，例如通过自助法（bootstrap）生成不同的训练集。

4. 不同的训练样本权重：在训练过程中，可以给不同的训练样本分配不同的权重，从而影响模型的学习过程。

我们将在下一章讲述其中的细节，我们先通过偏差-方差分解（bias-variance decomposition）来加深对泛化（generalization）能力的理解，这将帮助我们理解集成方法。

2. 偏差方差分解

这是机器学习中一个重要的概念，用于分析模型的泛化能力。决策树在某些情况下可能会出现过拟合（高方差）或欠拟合（高偏差）。通过偏差-方差分解，可以更好地理解模型的性能，并采取措施（如剪枝、集成学习）来优化模型。



过于简单的模型会欠拟合数据（图1），而过于复杂的模型会过拟合（图2）。

通过偏差-方差分解，我们可以量化模型的偏差和方差，从而更好地理解模型的泛化能力。

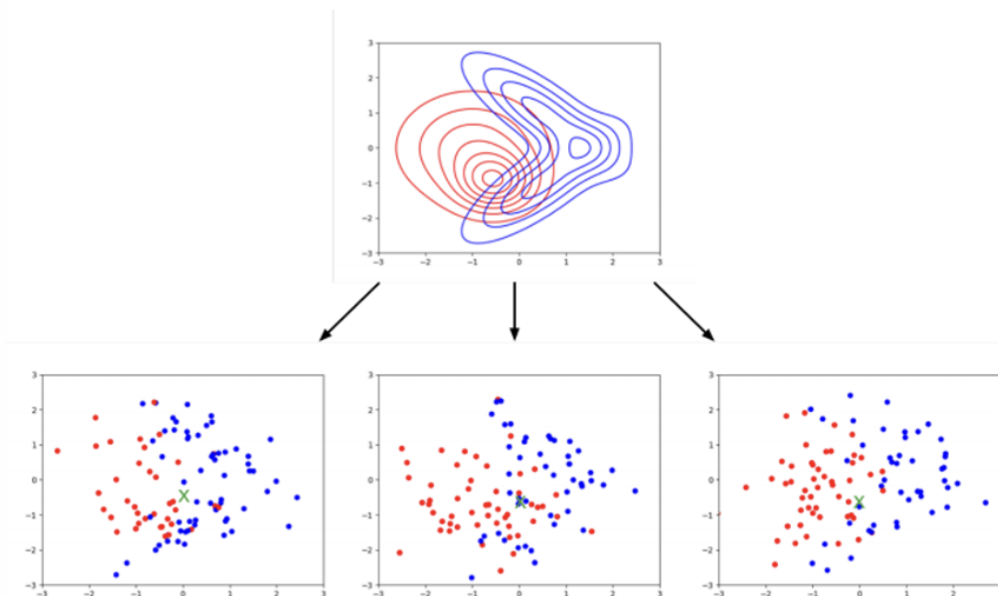
偏差：模型预测值与真实值之间的系统性误差。高偏差通常意味着模型过于简单，无法捕捉数据的真实模式。

方差：模型预测值在不同训练集上的波动程度。高方差通常意味着模型过于复杂，对训练数据的噪声过于敏感，导致过拟合。

2.1 基本设置

我们先假设：训练集 D 由从单一数据生成分布 p_{sample} 中独立同分布（ $i.i.d$ ）采样的对 (x_i, t_i) 组成。选择一个固定的查询点 x （用绿色 x 表示）。

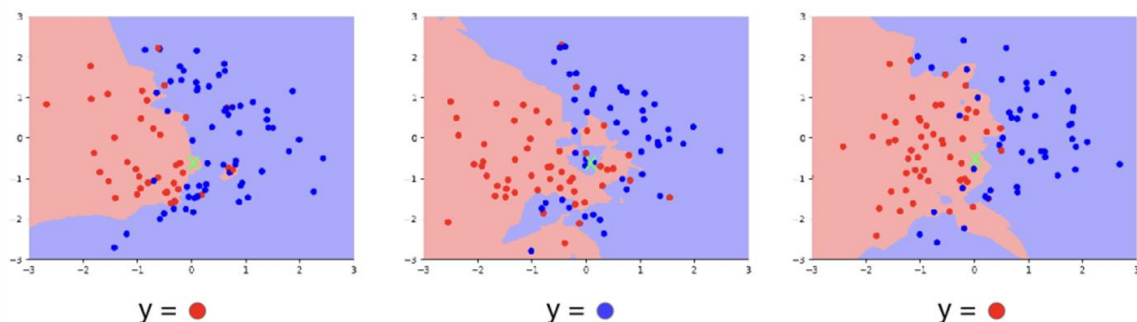
考虑一个实验，其中我们从 p_{sample} 独立采样许多训练集。



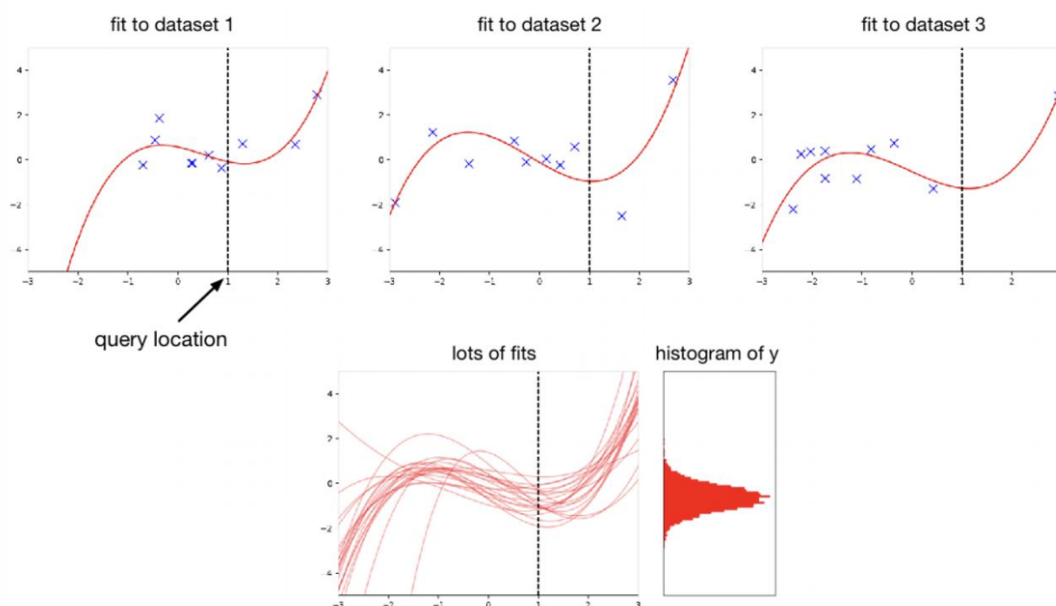
上图：展示了数据生成分布 p_{sample} 的等高线图，表示数据的潜在分布。

下图：展示了从同一分布中独立采样的三个不同的训练集，每个训练集包含红色和蓝色的点，代表不同的类别。

下一步：在每个训练集上运行学习算法，并计算在查询点 x 的预测 y 。
可以将 y 视为一个随机变量，其随机性来自于训练集的选择。
分类准确性由 y 的分布决定。



下图展示了多个模型在查询点 x 上的预测结果。



偏差：模型预测值与真实值之间的系统性误差。在图中，这可以体现为模型预测的平均值与目标变量 y 的真实分布之间的差异。

方差：模型预测在不同数据集上的波动程度。在图中，这可以体现为不同模型在查询点 x 上预测值的分散程度。

通过在多个数据集上训练模型并评估其在固定查询点 x 上的预测，我们可以量化模型的方差。

由于 y 是一个随机变量，我们可以讨论它的期望（均值）、方差等统计特性，这有助于我们理解模型的泛化能力和预测稳定性。

理想的模型应该在偏差和方差之间取得平衡，以确保在新数据上的预测既准确又稳定。

具体步骤如下：

1. 选择一个固定的查询点 x 作为评估模型性能的基准点。
2. 重复以下步骤：
 - 从数据生成分布 p_{sample} 中独立同分布（i.i.d.）地采样一个随机训练数据集 D 。
 - 在数据集 D 上运行学习算法，以获得在 x 处的预测 y 。
 - 从条件分布 $p(t | x)$ 中采样真实目标 t 。
 - 计算损失 $L(y, t)$ 。

注意： y 独立于 t ，即模型的预测与真实目标是独立的。

在 x 处损失的分布，其期望为 $E[L(y, t) | x]$ 。

对于每个查询点 x ，期望的损失是不同的。我们感兴趣的是最小化关于 $x \sim p_{sample}$ 的这个期望值。

2.2 贝叶斯最优性 (Bayes Optimality)

给定平方误差损失函数 $L(y, t) = \frac{1}{2}(y - t)^2$ ，假设我们知道条件分布 $p(t | x)$ ，即给定 x 时 t 的概率分布。我们需要选择一个值 y 来进行预测。

声明： $y_* = E[t | x]$ 是最好的预测，其中 y_* 是 t 给定 x 时的条件期望。

$$\begin{aligned} \text{证明如下: } \mathbb{E}[(y - t)^2 | x] &= \mathbb{E}[y^2 - 2yt + t^2 | x] \\ &= y^2 - 2y\mathbb{E}[t | x] + \mathbb{E}[t^2 | x] \\ &= y^2 - 2y\mathbb{E}[t | x] + \mathbb{E}[t | x]^2 + \text{Var}[t | x] \\ &= (y - y_*)^2 + \text{Var}[t | x] \end{aligned}$$

因此贝叶斯最优性的公式： $\mathbb{E}[(y - t)^2 | x] = (y - y_*)^2 + \text{Var}[t | x]$

第一项： $(y - y_*)^2$ 是非负的，可以通过设置 **Error: Missing superscript or subscript argument** 使其为0。

第二项： $\text{Var}[t | x]$ 对应于目标的固有不可预测性或噪声，称为贝叶斯误差。这部分误差是不可避免的，因为它是由数据本身的随机性引起的。

贝叶斯最优性表明，对于任何学习算法，使用条件期望作为预测值是最优的。如果一个算法能够实现这一点，那么它就是贝叶斯最优的。

贝叶斯误差项不依赖于 y ，这意味着无论我们选择什么预测值，这部分误差都是存在的。

选择一个单一值 y_* 基于 $p(t | x)$ 的过程是决策理论的一个例子，它展示了如何在给定信息下做出最优决策。

将 y 视为一个随机变量，其随机性来自于数据集的选择。

通过一系列代数变换，进一步分解为：

$$\begin{aligned} \mathbb{E}[(y - t)^2 | x] &= \mathbb{E}[(y - y_*)^2] + \text{Var}[t | x] \\ &= \mathbb{E}[y_*^2 - 2y_*y + y^2] + \text{Var}(t) \\ &= y_*^2 - 2y_*\mathbb{E}[y] + \mathbb{E}[y^2] + \text{Var}(t) \\ &= y_*^2 - 2y_*\mathbb{E}[y] + \mathbb{E}[y]^2 + \text{Var}(y) + \text{Var}(t) \\ &= (y_* - \mathbb{E}[y])^2 + \text{Var}(y) + \text{Var}(t) \end{aligned}$$

其中 $(y_* - \mathbb{E}[y])^2$ 是偏差的平方 (bias²)。

$\text{Var}(y)$ 是方差 (variance)。

$\text{Var}(t)$ 是贝叶斯误差 (Bayes error)。

$$\mathbb{E}[(y - t)^2] = \underbrace{(y_* - \mathbb{E}[y])^2}_{\text{bias}} + \underbrace{\text{Var}(y)}_{\text{variance}} + \underbrace{\text{Var}(t)}_{\text{Bayes error}}$$

我们将期望损失分解为三个部分：偏差的平方 (bias²)、方差 (variance) 和贝叶斯误差 (Bayes error)。

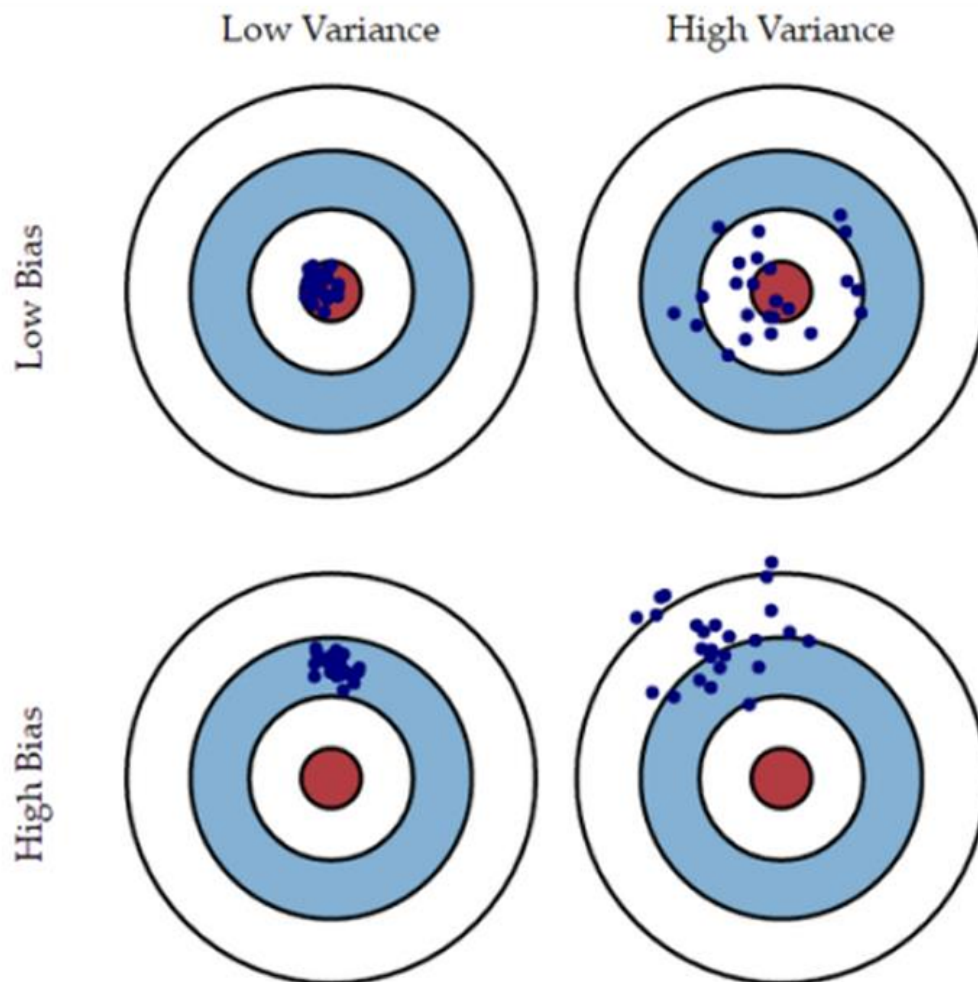
偏差 (Bias)：预测值与真实值之间的系统性误差，反映了模型的欠拟合 (underfitting)。

方差 (Variance)：模型预测值的波动性，反映了模型的过拟合 (overfitting)。

贝叶斯误差 (Bayes error)：目标变量的固有不可预测性或噪声，是模型无法减少的误差部分。

尽管这种分析仅适用于平方误差损失，但我们通常将“偏差”和“方差”松散地用作“欠拟合”和“过拟合”的同义词。这种分解帮助我们在模型选择和优化时做出更明智的决策，以提高模型的泛化能力和预测准确性。

下图用飞镖的比喻来形象地解释了偏差 (Bias) 和方差 (Variance) 对模型预测性能的影响。



四个飞镖靶，每个靶上都有不同分布的飞镖点，代表模型在不同情况下的预测结果。

左上角（低偏差，低方差）：飞镖点集中在靶心附近，表示模型预测准确且稳定。

右上角（低偏差，高方差）：飞镖点分布分散但都靠近靶心，表示模型预测稳定但不够准确。

左下角（高偏差，低方差）：飞镖点集中在偏离靶心的位置，表示模型预测不准确但稳定。

右下角（高偏差，高方差）：飞镖点分散且偏离靶心，表示模型预测既不准确也不稳定。

我们再回顾一下：

偏差：模型预测值与真实值之间的系统性误差。高偏差意味着模型欠拟合，即模型过于简单，无法捕捉数据的真实模式。

方差：模型预测值在不同训练集上的波动程度。高方差意味着模型过拟合，即模型过于复杂，对训练数据的噪声过于敏感。

理想的模型应该在偏差和方差之间取得平衡，既不欠拟合也不过拟合。我们在评估模型时应该考虑所有可能的查询点，而不仅仅是单个点的表现。